EXPERIMENT 15

IMPLEMENTATION OF BLOWFISH ALGORITHM

AIM:

To implement the Blowfish symmetric key encryption algorithm in C and perform encryption and decryption of a 64-bit data block using a secret key.

ALGORITHM:

Key Expansion:

1. Initialize the P-array and four S-boxes.
2. XOR the secret key with the entries of the P-array.
3. Encrypt an all-zero 64-bit block using the Blowfish encryption function.
4. Replace the P-array entries with the resulting encrypted values.
5. Continue the encryption process to replace all S-box entries.

Encryption:

1. Divide the 64-bit plaintext into two 32-bit halves, L and R.
2. Perform 16 Feistel rounds.
3. In each round:
4. L = L XOR P[i]
5. R = F(L) XOR R
   Swap L and R

6. After the 16 rounds, undo the final swap.
7. Perform:
8. R = R XOR P[16]
   L = L XOR P[17]

9. Combine L and R to obtain the ciphertext.

Decryption:

1. Divide the 64-bit ciphertext into two 32-bit halves, L and R.
2. Use the P-array entries in reverse order.
3. Perform the 16 Feistel rounds.
4. Undo the final swap.
5. Perform the final XOR operations with the P-array.
6. Combine L and R to obtain the original plaintext.

PROGRAM:

```
#include <stdio.h>

#include <stdint.h>

#include <string.h>


uint32_t P[18] = {

    0x243F6A88, 0x85A308D3,

    0x13198A2E, 0x03707344,

    0xA4093822, 0x299F31D0,

    0x082EFA98, 0xEC4E6C89,

    0x452821E6, 0x38D01377,

    0xBE5466CF, 0x34E90C6C,

    0xC0AC29B7, 0xC97C50DD,

    0x3F84D5B5, 0xB5470917,

    0x9216D5D9, 0x8979FB1B

};


uint32_t S[4][256];


/* Initialize S-boxes */

void init_sboxes()

{

    uint32_t x = 0x13579BDF;
```

```
    int i, j;

    for (i = 0; i < 4; i++)
    {
        for (j = 0; j < 256; j++)
        {
            x = x * 1664525u + 1013904223u;
            S[i][j] = x;
        }
    }
}

/* Blowfish F-function */
uint32_t F(uint32_t x)
{
    uint32_t a, b, c, d;

    a = (x >> 24) & 0xFF;
    b = (x >> 16) & 0xFF;
    c = (x >> 8) & 0xFF;
    d = x & 0xFF;

    return ((S[0][a] + S[1][b])
```

```
            ^ S[2][c])
            + S[3][d];
}

/* Blowfish encryption */
void encrypt(uint32_t *L, uint32_t *R)
{
    uint32_t temp;
    int i;

    for (i = 0; i < 16; i++)
    {
        *L = *L ^ P[i];
        *R = *R ^ F(*L);

        temp = *L;
        *L = *R;
        *R = temp;
    }

    /* Undo final swap */
    temp = *L;
    *L = *R;
```

```
    *R = temp;

    *R = *R ^ P[16];
    *L = *L ^ P[17];
}

/* Blowfish decryption */
void decrypt(uint32_t *L, uint32_t *R)
{
    uint32_t temp;
    int i;

    for (i = 17; i > 1; i--)
    {
        *L = *L ^ P[i];
        *R = *R ^ F(*L);

        temp = *L;
        *L = *R;
        *R = temp;
    }

    /* Undo final swap */
```

```
    temp = *L;
    *L = *R;
    *R = temp;

    *R = *R ^ P[1];
    *L = *L ^ P[0];
}

/* Key expansion */
void key_schedule(const unsigned char *key, int len)
{
    uint32_t data;
    uint32_t L = 0;
    uint32_t R = 0;

    int i, j;

    /* XOR key with P-array */
    for (i = 0; i < 18; i++)
    {
        data = 0;

        for (j = 0; j < 4; j++)
```

```
    {
        data = (data << 8)
                | key[(i * 4 + j) % len];
    }

    P[i] ^= data;
}

/* Encrypt zero block to replace P-array */
for (i = 0; i < 18; i += 2)
{
    encrypt(&L, &R);

    P[i] = L;
    P[i + 1] = R;
}

/* Replace S-box values */
for (i = 0; i < 4; i++)
{
    for (j = 0; j < 256; j += 2)
    {
        encrypt(&L, &R);
```

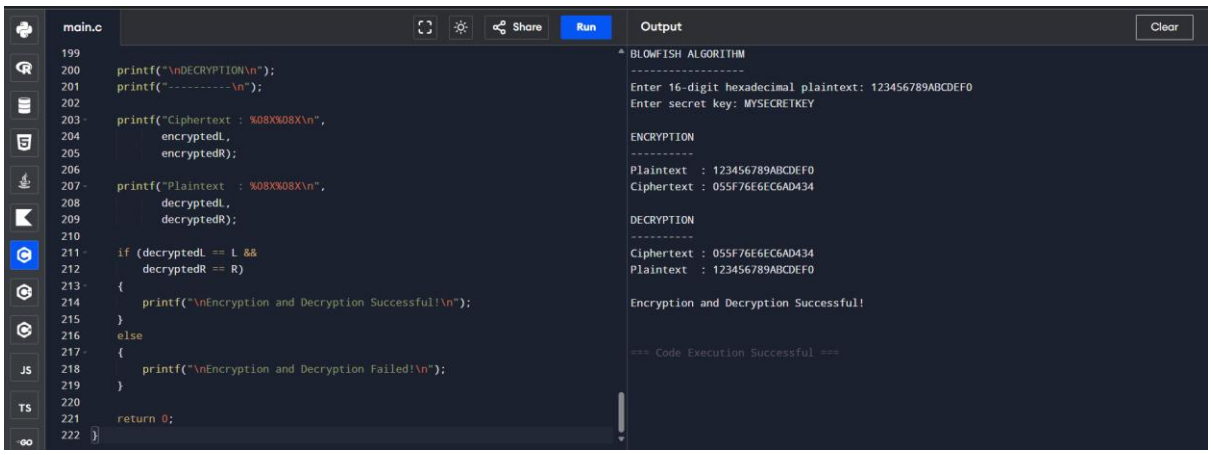
```
            S[i][j] = L;
            S[i][j + 1] = R;
        }
    }
}

int main()
{
    uint32_t L, R;
    uint32_t encryptedL, encryptedR;
    uint32_t decryptedL, decryptedR;

    char key[57];

    printf("BLOWFISH ALGORITHM\n");
    printf("------------------\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%8x%8x", &L, &R);

    printf("Enter secret key: ");
    scanf("%56s", key);
```

```
/* Initialize S-boxes */
init_sboxes();

/* Generate subkeys */
key_schedule(
    (unsigned char *)key,
    strlen(key)
);

/* Store plaintext */
encryptedL = L;
encryptedR = R;

/* Encryption */
encrypt(
    &encryptedL,
    &encryptedR
);

printf("\nENCRYPTION\n");
printf("----------\n");

printf("Plaintext  : %08X%08X\n",
```

```
        L, R);

printf("Ciphertext : %08X%08X\n",
        encryptedL,
        encryptedR);

/* Decryption */
decryptedL = encryptedL;
decryptedR = encryptedR;

decrypt(
    &decryptedL,
    &decryptedR
);

printf("\nDECRYPTION\n");
printf("----------\n");

printf("Ciphertext : %08X%08X\n",
        encryptedL,
        encryptedR);

printf("Plaintext  : %08X%08X\n",
```

```
        decryptedL,
        decryptedR);


    if (decryptedL == L &&
        decryptedR == R)
    {
        printf("\nEncryption and Decryption Successful!\n");
    }
    else
    {
        printf("\nEncryption and Decryption Failed!\n");
    }


    return 0;
}
```

OUTPUT:



RESULT: Thus, the Blowfish encryption and decryption algorithm was successfully implemented in C. The given 64-bit plaintext was encrypted using a secret key to produce the ciphertext.